

Chapter 1: Programming Foundations

How to read these notes

- **CORE** Review of EECS 280 material, needed constantly for projects and exams.
- **PROJECT USE** Good to know, mostly there to help you write better project code. Not something you are forced to use.
- **NOT REQUIRED** Beyond the scope of the class. Boxed with a dashed red border. Read it if you are curious, you will not be tested on it.

The whole chapter is review of EECS 280 concepts, provided mainly to help with projects.

1.1 Pointers and References **CORE**

1.1.1 Program Execution and the Machine Model

- The **machine model** is a way to visualize how code behaves at runtime.
- Execution starts at `main()`.

```
int main() {  
    int x = 280;  
    int y = x;  
    x = 281;  
    y = x;  
} // main()
```

- `int x = 280;` creates an integer object somewhere in memory. Its memory location is its **address**.

```
variable  
  \  
   -> x +-----+  
       | 280 |    <- address 0x7fff492414a4  
       +-----+
```

- You do not control which addresses your objects get. The operating system decides the exact locations at runtime.

Remark: the address-of operator

- The ampersand `&` placed next to an existing object gives its address.
- Do not confuse this with declaring a reference, which also uses `&`.

```
int x = 280;  
std::cout << &x << std::endl; // sample output: 0x7fff492414a4
```

- Addresses are printed in **hexadecimal** (base 16): leading `0x`, then digits `0-9` and letters `a-f`, where `a=10`, `b=11`, `c=12`, `d=13`, `e=14`, `f=15`.
- Address size depends on the machine: 64-bit machine = 64 bits (8 bytes) per address; 32-bit machine = 32 bits (4 bytes).

- `int y = x;` copies the **value** stored in `x` into new memory allocated for `y`.
- Important: `y = x` does **not** make `y` refer to the same object as `x`. Two separate objects exist.

```
x +-----+      y +-----+  
| 280 | 0x7fff492414a4 | 280 | 0x7ffdaddeb434  
+-----+      +-----+
```

- `x = 281;` updates only the memory of `x`. `y` still holds 280.
- `y = x;` copies again, so `y` becomes 281 as well.

1.1.2 References and Reference Semantics

- **Value semantics** (the default in C++): assignment sets the *value* of the target object.
- **Reference semantics**: assignment changes *which object* the variable refers to.

```
y = x (value semantics)    -> value of y is set to value of x  
y = x (reference semantics) -> the object y refers to becomes the object x refers to
```

- In C++, reference semantics is only supported **at initialization**. Place `&` to the left of the variable name.

```
int x = 280;  
int& y = x;                // y is a reference to x (an alias)  
x = 281;  
std::cout << y << std::endl; // prints 281
```

- A reference is just an alternative name for an existing object. Changing either name changes the same object in memory.

```
x,y +-----+      x = 281      x,y +-----+  
| 280 | 0x...4a4  ----->    | 281 | 0x...4a4  
+-----+      +-----+
```

- A reference **cannot be reassigned** once created. It stays bound to the same object until it goes out of scope.

Main use case: function parameters

```
void increment(int counter) { // pass by value
    counter += 1;
} // increment()

int main() {
    int c = 280;
    increment(c);
    std::cout << c << std::endl; // prints 280 (a copy was incremented)
} // main()
```

- The copy goes out of scope when the function returns, so `c` never changes.

```
void increment(int& counter) { // pass by reference
    counter += 1;
} // increment()

int main() {
    int c = 280;
    increment(c);
    std::cout << c << std::endl; // prints 281
} // main()
```

- If you want a function to modify an object, you must pass it **by reference**.
- Even when you do not want modification, passing a large object by **const reference** avoids the cost of copying it.

```
void func(const BigObject& obj) { // pass by const reference
    // do work...
} // func()

int main() {
    BigObject my_big_object{};
    func(my_big_object); // not copied, because it is passed by reference
} // main()
```

Remark: framework for choosing value vs reference

1. If a parameter should be **modified** by the function, pass by reference.
2. Else, if the parameter is a **primitive type** (int, char, etc.), pass by value.
3. Else, if it is an **object**, pass by const reference.

Why not pass primitives by reference? Passing by reference is really passing a pointer behind the scenes. That adds an unnecessary level of indirection and takes extra stack space, since primitives (int 4 bytes, double 8 bytes, char and bool 1 byte) are no larger than a pointer (8 bytes on a 64-bit machine).

1.1.3 Pointers

- Objects get their addresses at runtime. Running the same program twice can place objects at different addresses.
- A **pointer** is an object that stores an address. Declare it with `*` to the left of the variable name.

```
int x = 281;
int* ptr = &x; // ptr stores the address of x
std::cout << ptr << std::endl; // sample output: 0x7ffeccc22c4c
```

- Every data type has a corresponding pointer type (`int*` is a pointer to `int`).
- Pointers are objects too, so they have addresses. `int**` (pointer to pointer to int) is perfectly valid.

```
int x = 281;
int* y = &x; // y stores the address of x
int** z = &y; // z stores the address of y
```

x	y	z
281	0x4104	0x4108
0x4104	0x4108	0x4110

- **Dereference** a pointer with `*` to access the object it points to.

```
int x = 281;
int* y = &x;
std::cout << *y << std::endl; // prints 281 (*y gets the value y points to, or x)
```

Two warnings about pointers

1. An uninitialized pointer holds an undefined value. Dereferencing it is **undefined behavior**.

```
int *x;
*x = 281; // undefined behavior
```

2. A pointer can outlive the object it points to, which is also undefined behavior.

```
int* get_address(int x) {
    return &x; // x goes out of scope after the function returns
} // get_address()

int main() {
    int val = 280;
    int* ptr = get_address(val);
    std::cout << *ptr << std::endl; // undefined behavior
} // main()
```

- The **null pointer** (`nullptr`) marks a pointer that does not point to a valid object. Any pointer type can be set to it.

```
int* x = nullptr;
double* y = nullptr;
char* z = nullptr;
```

- Dereferencing `nullptr` is undefined behavior, same as a default-initialized pointer.

1.2 Address Space and Dynamic Memory CORE

- Where memory is allocated depends on the kind of object: scope-bound local variables go in one region, dynamically created objects go in another.
- Every running program is a **process**, and each process has its own memory layout (kept separate by the OS using virtual memory, beyond the scope of the class).
- The **address space** of a process is the range of valid addresses it can access.

```
high addresses
+-----+
|  stack  |   local variables, grows downward
+-----+
|  heap   |   dynamically allocated objects
+-----+
| static  |
+-----+
|  text   |
+-----+
low addresses
```

- **Stack**: holds local variables. Grows downward (first allocation gets the highest address). Allocation and deallocation are **automatic**: space is reserved on declaration and released when the variable goes out of scope.
- **Heap**: holds dynamically allocated objects. Allocated **explicitly** by the programmer and not tied to a scope. Must also be deallocated explicitly.
- Failing to deallocate causes a **memory leak**: the memory stays reserved indefinitely (until something like a shutdown wipes it) and can exhaust available memory.

new and delete

```
int main() {
    int* ptr = new int{281};
    std::cout << *ptr << std::endl; // prints 281
    delete ptr;
} // main()
```

- The lifetime of a dynamic object starts at `new` and ends at `delete`.
- `new` does three things:
 1. Allocates space for an object of the given type on the heap.
 2. Initializes the object with the given initialization expression.
 3. Evaluates to the **address** of the new object.

```
stack                heap
+-----+           +-----+
| 0x5370 | ptr -----> | 281 | 0x5370
+-----+           +-----+
```

- `ptr` itself is a local variable on the stack that points to heap memory.
- `delete ptr;` deletes the **object that `ptr` points to**, not `ptr` itself. `ptr` remains on the stack.
- `delete` may only be used on objects created with `new`. Anything else is undefined behavior.

Remark: dynamic arrays

An **array** is a fixed-size collection of objects of the same type, stored contiguously in memory. Heap arrays use `new[]` and `delete[]`.

```
int* arr = new int[5]; // array of 5 integers
for (int i = 0; i < 5; ++i) {
    arr[i] = i; // fills with [0, 1, 2, 3, 4]
} // for i
delete[] arr; // note the extra []
```

Remark: segmentation faults (segfault / SIGSEGV)

A runtime error that happens when a program touches memory it is not allowed to access. Common causes:

- a. dereferencing a null or invalid pointer, e.g. `int* ptr = nullptr; *ptr = 281;`
- b. accessing memory outside the bounds of an array
- c. writing to read-only memory
- d. using a pointer that has already been freed
- e. stack overflow from infinite or very deep recursion (chapter 5)

The OS memory management unit detects the invalid access and sends a `SIGSEGV` signal, which forcibly kills the process.

1.3 Compound Objects CORE

1.3.1 Primitive Data Types

You do **not** need to memorize every type with its size and value range. Knowing the common ones is still useful, especially for keeping memory usage low.

- **int**: typically 4 bytes, holds about -2,147,483,648 (-2^{31}) to 2,147,483,647 ($2^{31} - 1$).
- **char**: 1 byte, stores a number where 0 to 127 map to ASCII characters (e.g. 'a' is 97, covered in chapter 16).
- **double**: stores decimal values, typically 8 bytes.

Type	Bytes	Minimum	Maximum
double	8	2.22507e-308	1.79769e+308
int	4	-2,147,483,648 (-2^{31})	2,147,483,647 ($2^{31} - 1$)
char	1	0	127
bool	1	N/A	N/A

- **Type modifiers** adjust size or range:
 - **unsigned** makes a type non-negative (an unsigned `int` holds 0 to 4,294,967,295, or $2^{32} - 1$).
 - **short** and **long** shrink or expand the range. A **short int** holds -32,768 to 32,767; a **long int** holds -2^{63} to $2^{63} - 1$.
 - Tradeoff: a larger range costs more memory.

Type	Bytes	Minimum	Maximum
unsigned long	8	0	18,446,744,073,709,551,615 ($2^{64} - 1$)
long	8	-2^{63}	$2^{63} - 1$
unsigned int	4	0	4,294,967,295 ($2^{32} - 1$)
int	4	-2^{31}	$2^{31} - 1$
unsigned short	2	0	65,535 ($2^{16} - 1$)
short	2	-32,768 (-2^{15})	32,767 ($2^{15} - 1$)

- **Fixed-width integers** (C++11, `<stdint.h>`) guarantee an exact size. `int32_t` is exactly 32 bits (4 bytes). Unsigned versions prepend `u`, e.g. `uint32_t`.

Type	Bytes	Minimum	Maximum
uint64_t	8	0	$2^{64} - 1$
int64_t	8	-2^{63}	$2^{63} - 1$
uint32_t	4	0	$2^{32} - 1$
int32_t	4	-2^{31}	$2^{31} - 1$
uint16_t	2	0	$2^{16} - 1$
int16_t	2	-2^{15}	$2^{15} - 1$
uint8_t	1	0	255 ($2^8 - 1$)
int8_t	1	-128 (-2^7)	127 ($2^7 - 1$)

- **size_t**: an unsigned integer type used for object sizes, array indexing, loop counting, and size storing.
 - Its size is architecture dependent: at least 32 bits on a 32-bit system, at least 64 bits on a 64-bit system.
 - In this class treat it as 8 bytes (0 to $2^{64} - 1$).
- Choosing a type: think about the range of values you need.
 - Too small a range causes **overflow** or **underflow**, where the value wraps around. Adding 1 to 2,147,483,647 in an `int32_t` wraps to -2,147,483,648.
 - Too large a range wastes memory.

Footnotes from the text: the real size of `int` varies between systems, so `int32_t` is used when exactly 4 bytes are needed. A `char` can hold values outside 0 to 127, but those do not represent meaningful characters.

1.3.2 Structs and Classes

- **User-defined types** let you build your own types out of the primitives. These are **compound objects**: structs and classes.

```
struct Student {  
    std::string name;  
    std::string username;  
    int32_t age;  
    int32_t id;  
};
```

- This is only the **definition** (a blueprint). Objects must be created explicitly:

```
Student s; // instantiates a Student, members are default initialized (junk values!)
```

- Default initialization of members is bad practice because members may hold indeterminate values.
- **Initializer list**, in the same order as the member declarations:

```
Student s1 = {"Bobby Tables", "btables", 21, 12345678};  
Student s2{"Bobby Tables", "btables", 21, 12345678};
```

- **Designated initializers**: a period, then the member name. Order must still follow the struct definition.

```
Student s{
    .name = "Bobby Tables",
    .username = "btables",
    .age = 21,
    .id = 12345678
};
```

- Access members with the dot operator `.`:

```
Student s{"Bobby Tables", "btables", 21, 12345678};
s.id = 87654321;
std::cout << s.id << std::endl; // prints 87654321
```

- Through a pointer, use `->` to dereference and access a member in one step:

```
Student* s_ptr = &s;
s_ptr->id = 87654321;
std::cout << s_ptr->id << std::endl; // prints 87654321
```

struct vs class

- Common misconception: that only classes support member functions. Not true.
- A C++ `class` is based on a C `struct`, and the two are essentially identical.
- **The one real difference**: members of a `class` are `private` by default, members of a `struct` are `public` by default.
- They are interchangeable if you set access levels manually, but the convention is: use `struct` when everything should be public, `class` everywhere else.

```
class String {
public:
    clear(); // public member functions the user can call
    erase(...); // clears the content of the string
    insert(...); // erases portion of string
    insert(...); // inserts another string into the original string
    replace(...); // replaces a portion of the string with new contents
private:
    char* cstr; // private member variables that store the data
    size_t size; // c-string holding the contents
    // size of the string object
};
```

- `public` members are accessible outside the class; `private` members are not.
- Making members `private` protects **invariants**. A user should not be able to set the size of "apple" to 563; `size` must stay consistent with the contents, so the class handles it internally.

Remark: abstract data types (ADTs)

- **Abstraction** separates *what* something does (the interface) from *how* it works (the implementation). The user only gets the essential information needed to use the object.
- Car analogy: you press the accelerator to go faster and the brake to slow down. You do not need to know the mechanics behind it.
- An **ADT** gives the user operations and their expected behaviors on stored data, without defining the implementation of those operations. Several ADTs are covered throughout the course.

1.3.3 Constructors

- A **constructor** initializes a class-type object and is called when the object is created.
- Same name as the class, and **no return type**.
- Use a **member-initializer list**: a colon after the parameter list, then members in the order they are declared, each initialized in parentheses or braces.

```
struct Student {
    std::string name;
    int32_t age;
    Student(std::string name_in, int32_t age_in)
        : name{name_in}, age{age_in} {}
};

Student s{"Alice", 21};
```

- Always initialize variables before use. For atomic types like `int` and `double`, default initialization does not set a predetermined value, it leaves whatever junk was already in that memory.
- If a class defines **no** constructors, the compiler supplies an implicit **default constructor** that default initializes each member.
- If you write any non-default constructor and still want a default one, you must write it yourself:

```
Student()
    : name{"Potato"}, age{281} {}
```

1.3.4 Member Variable Organization PROJECT USE

This is mostly EECS 370 material. You are not expected to know struct alignment, but ordering your members well lowers memory usage on projects, which can affect your score.

- Same members, different order, different total size:

```

struct StudentA {
    std::string name;
    double eecs_281_score;
    int64_t enrollment_timestamp;
    char letter_grade;
    bool has_passed;
    bool has_applied_for_ia;
};

struct StudentB {
    bool has_passed;
    double eecs_281_score;
    char letter_grade;
    std::string name;
    bool has_applied_for_ia;
    int64_t enrollment_timestamp;
};

```

```

std::cout << sizeof(StudentA) << std::endl; // prints 56
std::cout << sizeof(StudentB) << std::endl; // prints 72

```

- The cause is **struct alignment** (beyond the scope of this class).
- **Simple rule:** list member variables in order of increasing or decreasing size. Doubles (8 bytes), then ints (4 bytes), then chars (1 byte), or the reverse.

Remark: this is a simplification. Member order also controls the order member constructors run, so a member that must be initialized before another should be listed first. Style matters too: unless you create many instances, the memory saved is not worth hurting readability or maintainability.

1.4 Operator Overloading CORE

- An **operator** is a symbol that performs a specific function. Every operator has a function-like name (+ is operator+).
- Operators are predefined for built-in types like `int` and `double`, but must be defined explicitly for custom types. This is **operator overloading**.
- Think of it as implementing a function that is invoked with a symbol instead of a name.
- Commonly overloaded operators:

```

operator+  operator-  operator*  operator/  operator=  operator<  operator>
operator<= operator>= operator++  operator--  operator==  operator!=  operator+=
operator-=  operator[]  operator()  operator%  operator^  operator<<  operator>>

```

- Example type used below:

```

struct Point {
    int32_t x;
    int32_t y;
    Point() // default constructor
        : x{0}, y{0} {}
    Point(int32_t x_in, int32_t y_in) // custom constructor
        : x{x_in}, y{y_in} {}
};

```

Overloading operator+ as a free function

Goal: `Point{3, 5} + Point{2, 6} = Point{5, 11}`.

Point	operator+	(const Point& lhs, const Point& rhs);
-----	-----	-----
return	operator	objects the operator acts upon
type	name	

```

Point operator+(const Point& lhs, const Point& rhs) {
    return Point{lhs.x + rhs.x, lhs.y + rhs.y};
} // operator+()

int main() {
    Point a{3, 5};
    Point b{2, 6};
    Point c = a + b; // c = (5, 11)
} // main()

```

- `lhs` = left-hand side, `rhs` = right-hand side. The `const` keeps the original Points from being modified.

Overloading as a member function

- Custom is to overload inside the type definition when possible. The exception is when the left-hand object is not the same type as the class.
- As a member function, the first argument (`lhs`) is **implicit**. You can use members directly: `lhs.x` becomes `x`, which means `this->x`.

```

struct Point {
    int32_t x;
    int32_t y;
    Point() : x{0}, y{0} {}
    Point(int32_t x_in, int32_t y_in) : x{x_in}, y{y_in} {}

    // lhs is implicit (lhs == *this)
    Point operator+(const Point& rhs) const {
        return Point{x + rhs.x, y + rhs.y};
    } // operator+()
};

```

Operators that modify the object (like +=)

- These must return an object **by reference**. Outside the class, return `lhs`:

```
Point& operator+=(Point& lhs, const Point& rhs) {
    lhs.x += rhs.x;
    lhs.y += rhs.y;
    return lhs;
} // operator+=()
```

- As a member function, return `*this`. `this` is an internal pointer to the current instance, so `*this` is the object itself.

```
Point& operator+=(const Point& rhs) {
    x += rhs.x;
    y += rhs.y;
    return *this;
} // operator+=()
```

- Returning `*this` by reference allows **chaining**: `a += b += c` is valid and sets `a` to `a + (b += c)`.

Overloading operator<<

- If the left-hand parameter is not the same type as the custom object, the overload **cannot** be a member function. `operator<<` is the classic case, since the left side is a stream.
- Without an overload you would write:

```
std::cout << "(" << pt.x << ", " << pt.y << ")" << std::endl;
```

- With an overload you can just write `std::cout << pt << std::endl;`.
- Take an `ostream` by reference and return the same `ostream` by reference:

```
std::ostream& operator<<(std::ostream& os, const Point& pt) {
    os << "(" << pt.x << ", " << pt.y << ")";
    return os;
} // operator<<()

int main() {
    Point a{3, 5};
    Point b{2, 6};
    std::cout << a + b << std::endl; // prints "(5, 11)"
} // main()
```

- Returning the stream by reference is what allows chaining. Without it, `cout << a + b` would work but `cout << a + b << endl` would not.

1.5 Function Objects and Comparators CORE

- Custom objects have no built-in ordering, so sorting them requires telling the program how to compare them.
- One option: overload `operator<`.

```
struct Student {
    std::string name;
    int32_t age;
    int32_t id;
    bool operator<(const Student& rhs) const {
        return age < rhs.age;
    } // operator<()
};
```

- Problem: `operator<` can only be overloaded **once** per object. If one step of an algorithm needs sorting by name and another by age, that does not work.
- Solution: a **comparator**, which is a **function object** (also called a **functor**): an object that overloads the function call operator `operator()`, so it can be called like a function.

```
struct Student {
    std::string name;
    int32_t age;
    int32_t id;
};

// comparator for Student
struct StudentComparator {
    bool operator() (const Student& lhs, const Student& rhs) const {
        return lhs.age < rhs.age;
    } // operator()()
};

int main() {
    Student s1{"Alice", 21, 12345678};
    Student s2{"Bob", 22, 87654321};
    StudentComparator comp; // create function object
    std::cout << comp(s1, s2) << std::endl; // prints whether s1 < s2 (true, 21 < 22)
} // main()
```

Multi-level tie breaking

Target ordering: by age first (younger first), ties broken by name (alphabetical), then by student ID (guaranteed unique).

```
struct StudentComparator {
    bool operator() (const Student& lhs, const Student& rhs) const {
        if (lhs.age == rhs.age) {
            if (lhs.name == rhs.name) {
                return lhs.id < rhs.id;    // names also equal, ID decides
            } // if
            return lhs.name < rhs.name;    // ages equal, name decides
        } // if
        return lhs.age < rhs.age;
    } // operator()()
};
```

```
Student s1{"Alice", 20, 12345678};
Student s2{"Alice", 21, 23456789};
Student s3{"Alice", 21, 34567890};
StudentComparator comp;
comp(s1, s2);    // true (younger age)
comp(s2, s3);    // true (same name and age, lower ID)
```

Other uses of function objects

- Functors work with many standard library algorithms (covered in chapter 11). Example: write a functor that returns true when an object meets a removal condition, then pass it to a library algorithm that removes matching elements in a range.
- Because a functor is a struct or class, it can **hold state** in member variables:

```
struct Student {
    std::string name;
    double exam_score;
};

struct ExamPass {
private:
    double threshold;
public:
    ExamPass(double thres_in) : threshold{thres_in} {}

    bool operator() (const Student& s) const {
        return s.exam_score > threshold;
    } // operator()()
};

int main() {
    Student s{"Alice", 45};
    ExamPass pass{50};    // threshold 50
    std::cout << pass(s) << std::endl;    // prints 0 (false)
    ExamPass pass2{40};    // threshold 40
    std::cout << pass2(s) << std::endl;    // prints 1 (true)
} // main()
```

1.6 Scope and Namespaces PROJECT USE

Good information to know, but not something you will directly work with in this class.

1.6.1 Scope

- The **scope** of an object is the region of code where its name can be used. Scope is decided at **compile time**, so using an out-of-scope name is a compile error.
- **Local (block) scope**: starts at the declaration, ends at the closing brace of the enclosing block.

```
int main() {
    // cannot use val here (not declared)
    {
        int32_t val = 281;
        // can use val here
    }
    // cannot use val here (out of scope)
} // main()
```

- Within a single scope, each name must refer to exactly one entity:

```
int32_t val = 281;
int32_t val = 370;    // not allowed
```

- An **inner scope** may reuse a name from an outer scope for a different entity. The inner scope then only sees its own version. This is typically discouraged.

```
int main() {
    int32_t val = 281;
    {
        int32_t val = 370;
        std::cout << val << std::endl;    // prints 370
    }
    std::cout << val << std::endl;    // prints 281
} // main()
```

- Variables declared in the introduction of a statement (such as a loop) are local to the block that statement introduces:


```
int32_t i = 281;
for (int32_t i = 0; i < 5; ++i) {
    std::cout << i << " ";
} // for i
// prints "0 1 2 3 4", not "281 281 281 281 281"
```

- **Class scope:** a member's scope covers the entire class no matter where it is declared. Each name can name only one member, variable or function.

```
class MyClass {
    int32_t sum;
    int32_t sum(int32_t a, int32_t b) { // not allowed, sum already names a member
        return a + b;
    } // sum()
};
```

- **Global scope:** declared outside any class or function. Runs from the point of creation to the end of the file. The `extern` keyword extends it to other files in a project (not needed for this class).

```
int32_t VAL = 281; // global variable

void func() {
    std::cout << VAL << std::endl;
} // func()

int main() {
    VAL = VAL + 1; // VAL is now 282
    func(); // prints 282
} // main()
```

1.6.2 Namespaces

Marked as extra material in the notes. NOT REQUIRED

- Problem: entities in the same scope cannot share a name. With large projects and external libraries, name collisions happen. If library A has a global `do_something()` and a library B upgrade adds its own `do_something()`, the project stops compiling.
- A **namespace** groups entities that would otherwise share the global scope into a smaller **namespace scope**.

```
namespace foo {
    void do_something() { /*...*/ }
} // namespace foo

namespace bar {
    void do_something() { /*...*/ }
} // namespace bar

int main() {
    foo::do_something(); // foo version
    bar::do_something(); // bar version
} // main()
```

- `::` is the **scope resolution operator**.
- Alternatively use the `using` keyword to pull in a whole namespace, or just specific names:

```
{ using namespace foo; do_something(); } // runs foo::do_something()
{ using bar::do_something; do_something(); } // runs bar::do_something()
```

- The entire C++ standard library lives in the `std` namespace, which is why `std::string` and `std::vector<>` need the `std::` prefix. Other libraries have their own, for example Boost uses the `boost` namespace.

Remark: never put `using namespace std;` in a header file. Anyone who includes your header is then forced to use every namespace you pulled in, which can break their code. The same applies to any namespace, not just `std`.

1.7 Ternary Operator CORE

- The **ternary (conditional) operator** shortens an if-else into one line:

```
<CONDITION> ? <EXPRESSION IF TRUE> : <EXPRESSION IF FALSE>
```

```
// original
bool passed_class(int32_t score) {
    if (score >= 70) { return true; }
    else { return false; }
} // passed_class()

// with ternary
bool passed_class(int32_t score) {
    return score >= 70 ? true : false;
} // passed_class()
```

- The part before `?` decides whether the value before the colon or after the colon is used.

```
// original
bool try_buy_item(double price, double willingness_to_pay, double& total_savings) {
    double savings_from_purchase;
    if (price < willingness_to_pay) {
        savings_from_purchase = willingness_to_pay - price;
    } // if
    else {
        savings_from_purchase = 0.0;
    } // else
    total_savings += savings_from_purchase;
    return price <= willingness_to_pay;
} // try_buy_item()

// with ternary
bool try_buy_item(double price, double willingness_to_pay, double& total_savings) {
    double savings_from_purchase = price < willingness_to_pay ? willingness_to_pay - price : 0.0;
    total_savings += savings_from_purchase;
    return price <= willingness_to_pay;
} // try_buy_item()
```

1.8 Common C++ Keywords

1.8.1 The const Keyword CORE

- A const variable cannot be modified.

```
const int32_t MAX_VALUE = 281;
MAX_VALUE = 5; // not allowed
```

- With pointers there are **two** things that could be modified:

```
int32_t* num = new int32_t{281};
*num = 280; // change the contents pointed to
int32_t* num2 = new int32_t{280};
num = num2; // change which address the pointer holds
```

- const **before** the *: the pointed-to value cannot change, but the pointer can be reassigned.

```
const int32_t* num = new int32_t{281};
*num = 280; // not allowed
```

- const **after** the *: the pointer cannot be reassigned, but the value can change.

```
int32_t* const num = new int32_t{281};
num = nullptr; // not allowed
```

- const in both places locks both:

```
const int32_t* const num = new int32_t{281};
*num = 280; // not allowed
num = nullptr; // not allowed
```

Remark: west const vs east const

West const (used in these notes) puts const to the left of what it modifies:

```
const int32_t val = 281; // const int
const int32_t& ref = val; // const int reference
const int32_t* ptr1 = &val; // pointer to const int
int32_t* const ptr2 = &val; // const pointer to int
const int32_t* const ptr3 = &val; // const pointer to const int
```

East const puts const to the right of what it modifies:

```
int32_t const val = 281;
int32_t const& ref = val;
int32_t const* ptr1 = &val; // int cannot be modified
int32_t* const ptr2 = &val; // pointer cannot be modified
int32_t const* const ptr3 = &val;
```

East const has one consistent rule: whatever is to the **left** of const is what cannot be modified. West const is more widespread but less consistent. Either is fine, but **be consistent**, especially inside an existing file or codebase. These notes use west const because it is more common in existing code.

The two uses of const on a member function

```
struct Student {
    std::string name;
    int32_t age;
    int32_t id;
    bool operator<(const Student& rhs) const {
        return age < rhs.age;
    } // operator<()
};
```

1. Passing by const reference: operator<(const Student& rhs)

- Gives direct access to the original object with no copy, which matters for large objects since copying costs time and memory.
- The const guarantees the function will not modify rhs.

```
bool operator<(const Student& rhs) const {
    rhs.age = 20;           // not allowed
    return age < rhs.age;
}
```

2. Const-qualified member function: the `const` after the parameter list

- Means the member function cannot modify any member variables of the object it is called on (`name`, `age`, `id`).

```
bool operator<(const Student& rhs) const {
    age = 20;               // not allowed
    return age < rhs.age;
}
```

- Why it matters:** non-const member functions can only be called by non-const objects. Const member functions can be called by **any** object.

```
struct Student {
    std::string name;
    int32_t age;
    int32_t id;
    bool operator<(const Student& rhs) {      // missing const
        return age < rhs.age;
    } // operator<()
};

void print_smaller_age(const Student& s1, const Student& s2) {
    if (s1 < s2) { std::cout << s1.age << std::endl; } // compile error
    else { std::cout << s2.age << std::endl; }
} // print_smaller_age()
```

- `s1` and `s2` are const, and the compiler cannot guarantee that a non-const `operator<` leaves them alone, so it rejects the call. Fix by marking the function `const`:

```
bool operator<(const Student& rhs) const { // now callable by const objects
    return age < rhs.age;
} // operator<()
```

- Rule of thumb: if a member function does not modify the object, mark it `const`.

1.8.2 The constexpr Keyword

Marked as extra material in the notes. **NOT REQUIRED**

- `constexpr` means **constant expression**: a value that is both constant and known at compile time.
- Used well, it moves work from runtime to compile time. It is also required in places where runtime values are not allowed, such as an array size:

```
constexpr size_t array_size = 281;
int32_t my_arr[array_size]; // array of size 281
```

- A `constexpr` **function** returns a compile-time constant if its arguments are compile-time constants. Otherwise it behaves like a normal function and computes at runtime.

```
constexpr double pi = 3.14159265358979323846;
constexpr double degrees_to_radians(double angle) {
    return angle * (pi / 180.0);
} // degrees_to_radians()

int main() {
    constexpr double radian1 = degrees_to_radians(57.296); // computed while compiling
    double degree_from_file = read_value_from_file();
    double radian2 = degrees_to_radians(degree_from_file); // computed during runtime
} // main()
```

- User-defined types can be built at compile time if their constructors and member functions are `constexpr`:

```
class Foo {
    int32_t i;
    double j;
public:
    constexpr Foo(int32_t i_in, double j_in) : i{i_in}, j{j_in} {}
    constexpr int32_t get_i() const { return i; }
    constexpr double get_j() const { return j; }
};

constexpr Foo add_foos(const Foo& f1, const Foo& f2) {
    return Foo{f1.get_i() + f2.get_i(), f1.get_j() + f2.get_j()};
} // add_foos()

int main() { // all done at compile time
    constexpr Foo f1{183, 2.8};
    constexpr Foo f2{281, 3.7};
    constexpr Foo f3 = add_foos(f1, f2); // Foo{464, 6.5}
} // main()
```

- In general, moving computation from runtime to compile time makes the program run faster, usually worth a longer compile.

1.8.3 The static Keyword **CORE**

- Three main uses: a variable inside a function, a variable inside a class or struct, and a global variable in one file of a multifile program.

- **Static local variable:** only one copy exists, and it keeps its value for the whole program.

```
int32_t foo() {
    static int32_t counter = 0;
    counter += 1;
    return counter;
} // foo()

int main() {
    std::cout << foo() << std::endl; // prints 1
    std::cout << foo() << std::endl; // prints 2
    std::cout << foo() << std::endl; // prints 3
} // main()
```

- **Static member variable:** shared by every instance of the object instead of one copy per object. Accessing it outside the class requires the scope resolution operator `::`.

```
struct Foo {
    static int32_t counter;
    void print_count() {
        counter += 1;
        std::cout << counter << std::endl;
    } // print_count()
};

int32_t Foo::counter = 0; // initializes counter for all Foo objects

int main() {
    Foo f1;
    Foo f2;
    f1.print_count(); // prints 1
    f2.print_count(); // prints 2
} // main()
```

- **Static global variable:** its scope is limited to the file it is defined in. Other files in the same project cannot access it.

1.8.4 The mutable Keyword PROJECT USE

Covered later in the class. You may see it on project 2.

- A const member function normally cannot modify any member variable. Sometimes you want an exception:

```
class ProjectScoreTracker {
    double project_average;
    ...
    int32_t request_counter; // other data
                          // counts how many times scores are requested
public:
    ProjectScoreTracker() : request_counter{0} {}
    void request_scores() const {
        // prints out project score data
        ++request_counter; // does not compile: const function modifying a member
    } // request_scores()
};
```

- Removing `const` is not a fix, because then a `const ProjectScoreTracker` could not call the function.
- Declaring the member `mutable` gives const member functions permission to modify **that member only**:

```
mutable int32_t request_counter; // now ++request_counter compiles inside a const function
```

- Other members (like `project_average`) still cannot be modified.
- Typical use: variables that hold **metadata** about the data rather than the data itself. The metadata can change while the invariant that the real data is unchanged still holds.

1.8.5 The explicit Keyword PROJECT USE

Covered later in the class. You may see it on project 2.

- A **conversion constructor** is a single-argument constructor whose argument type differs from the class type.

```
class Foo {
    int32_t foo_x;
public:
    Foo(int32_t input) : foo_x{input} {}
};

Foo myFoo{3}; // sets foo_x to 3
```

- The danger: the compiler can use such a constructor for an **implicit conversion**, without the programmer asking.

```
Foo myFoo = 3; // compiler silently converts 3 into a Foo
```

- Unwanted implicit conversions are among the hardest bugs to track down. Marking the constructor `explicit` forces the programmer to call it directly:

```
class Foo {
    int32_t foo_x;
public:
    explicit Foo(int32_t input) : foo_x{input} {}
};

Foo myFoo{3}; // okay, constructor explicitly called
Foo myFooToo = 3; // compile error, implicit conversion blocked
```

Worked example of the bug

```
class CustomString {
public:
    CustomString(int32_t n); // constructs a CustomString with size n
    CustomString(const char* p); // initializes contents from a char*
};

void print(CustomString c); // prints the CustomString

print("EECS"); // fine, prints EECS
print(281); // programmer meant print("281") but forgot the quotes
```

- The compiler sees an `int32_t` where a `CustomString` is needed, finds the non-explicit single-argument constructor, and implicitly builds an **empty CustomString of size 281**. No error is issued, and the program prints an empty string of size 281.
- Marking `explicit CustomString(int32_t n);` prevents this.

1.8.6 The using Keyword and Type Aliases CORE

- Type aliasing** gives an alternative name to a type, which helps when type names get long.

```
std::pair<int32_t, std::pair<std::string, std::string>> my_student; // painful to retype
```

- Syntax: `using <ALIAS> = <ORIGINAL TYPE NAME>;`

```
using StudentPair = std::pair<int32_t, std::pair<std::string, std::string>>;
StudentPair my_student;
```

- A single type can have multiple aliases:

```
using Uniqname = std::string;
using FirstName = std::string;
using LastName = std::string;
std::pair<Uniqname, std::pair<FirstName, LastName>> myStudent = ...
```

1.8.7 The auto Keyword and Automatic Type Deduction CORE

- Since C++11, `auto` lets the compiler deduce a variable's type from its initializer. A variable declared `auto` **must** be initialized.

```
int i = 281; // i is int
auto j = 281; // j is int
auto& k = i; // k is int&
```

- `auto` is a double-edged sword: it cleans up code but can hide a type that is hard to find (for example an unclear function return type defined far away).
- Use type deduction to make code **clearer or safer**, not just to avoid typing. Iterator types in a loop are a fine use. Important types that carry information for the reader should be written out.

1.8.8 The decltype Keyword and Function Return Type Deduction

Marked as extra material in the notes. NOT REQUIRED

- Problem: a templated function whose return type depends on its parameters.

```
template <typename T1, typename T2>
??? add(T1 a, T2 b) { return a + b; }
```

- C++11 solution: **trailing return type** syntax plus `decltype`. Normally the return type comes first:

```
int32_t add(int32_t x, int32_t y) { return x + y; }
auto add(int32_t x, int32_t y) -> int32_t { return x + y; } // trailing return type
```

```
template <typename T1, typename T2>
auto add(T1 a, T2 b) -> decltype(a + b) { return a + b; }
```

- `decltype` queries the type of an expression at compile time, commonly for types that depend on template parameters.
- The old syntax cannot be used here, because the compiler parses the leading return type before the parameters are declared, so `a` and `b` are not in scope yet:

```
// does not compile
template <typename T1, typename T2>
decltype(a + b) add(T1 a, T2 b) { return a + b; }
```

- C++14 added `decltype(auto)` and automatic return type deduction:

```
template <typename T1, typename T2>
decltype(auto) add(T1 a, T2 b) { return a + b; }
```

1.9 Templates and Generic Programming PROJECT USE

You can pass this class without knowing how to write templates. You will never be forced to implement something templated yourself, but templates show up in starter files, so it is worth refreshing.

1.9.1 Introduction to Templates

- C++ is **statically typed**: variable types must be known at compile time. That raises the question of how the same code can serve different types.

```
void swap(int32_t& a, int32_t& b) {
    int32_t temp = a;
    a = b;
    b = temp;
} // swap()
```

- This only works for `int32_t`. A reference to `int32_t` cannot bind to a `double`.
- Templates** allow **generic programming**: code that works for many types. The compiler generates the type-specific version as needed.

```
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
} // swap()
```

- `T` is the **type parameter**. The compiler deduces `T` from the call arguments and substitutes the real type.
- The compiler must see the template definition **before** it is used.

```
int32_t a = 5;
int32_t b = 6;
swap(a, b);
```

```
// compiler deduces T is int32_t      // and for two doubles it deduces T is double
void swap(int32_t& a, int32_t& b) {    void swap(double& a, double& b) {
    int32_t temp = a;                  double temp = a;
    a = b;                             a = b;
    b = temp;                           b = temp;
}
```

- A function template can take more than one type parameter:

```
template <typename T1, typename T2>
void print_two_types(T1& a, T2& b) {
    std::cout << a << " " << b << std::endl;
} // print_two_types()

int main() {
    std::string a = "EECS";
    int32_t b = 281;
    print_two_types(a, b);    // T1 is std::string, T2 is int32_t, prints "EECS 281"
} // main()
```

Class templates

```
// only supports integers          // templated version
struct CustomPair {               template <typename T1, typename T2>
    int32_t first;                 struct CustomPair {
    int32_t second;                T1 first;
    CustomPair()                  T2 second;
    : first{}, second{} {}        CustomPair() : first{}, second{} {}
    CustomPair(int32_t f, int32_t s) CustomPair(T1 f, T2 s)
    : first{f}, second{s} {}      : first{f}, second{s} {}
};                                };

```

- When creating an instance of a templated object, the types **must be specified**, since the compiler has nothing to deduce them from:

```
CustomPair cp;                    // error, T1 and T2 unknown
CustomPair<int32_t, double> cp;    // correct, first is int32_t and second is double
```

- Class templates are essential for **container classes**. `std::vector<int>` uses a class template to say the vector holds ints. The C++ **standard template library (STL)** implements common data structures and algorithms this way.

1.9.2 Variadic Templates

Marked as extra material in the notes. NOT REQUIRED

- **Variadic templates** take a variable number of arguments. They are defined **recursively**: process some arguments, then recurse on the rest. A “base case” version is required for when nothing is left.

```
template <typename T>
void print(T last_elt) {                // base case
    std::cout << last_elt << std::endl;
} // print()

template <typename T, typename... Args>
void print(T first_elt, Args... rem_elts) { // Args... is a parameter pack
    std::cout << first_elt << " ";
    print(rem_elts...);                 // parameter unpacking
} // print()

int main() {
    print("a", 5, 1 + 2, "apple");      // prints "a 5 3 apple"
} // main()
```

Trace of `print("a", 5, 1 + 2, "apple")`:

```
print("a", 5, 1+2, "apple") -> first_elt = "a",   prints "a",   calls print(5, 1+2, "apple")
print(5, 1+2, "apple")      -> first_elt = 5,      prints "5",   calls print(1+2, "apple")
print(1+2, "apple")         -> first_elt = 3,      prints "3",   calls print("apple")
print("apple")              -> base case, last_elt = "apple", prints "apple"
final output: "a 5 3 apple"
```

- Without the base case the compiler would not know what to do when the arguments run out, and the code would not compile.
- Variadic functions can also handle several arguments per call:

```
template <typename T1, typename T2>
void print_pairs(T1 a, T2 b) {
    std::cout << a << " " << b << std::endl;
} // print_pairs()

template <typename T1, typename T2, typename... Args>
void print_pairs(T1 first, T2 second, Args... args) {
    std::cout << first << " " << second << std::endl;
    print_pairs(args...);
} // print_pairs()

int main() {
    print_pairs("a", 5, 1 + 2, "apple"); // output: a 5 then 3 apple
} // main()
```

- If several arguments are handled at once, every possible base case needs a match. With an odd number of arguments:

```
print_pairs("a", 5, 1 + 2, "apple", "sauce");
// print_pairs(std::string, int, <int, std::string>); second definition
// print_pairs(int, std::string, <std::string>);      second definition
// print_pairs(std::string);                          no matching function, compile error
```

Remark: a practical use is printing a priority queue, which supports many template parameters. Templating just the element type is not enough, so variadic templates absorb the rest:

```
template <typename Type, typename... Params>
void print_priority_queue(std::ostream& os, const std::priority_queue<Type, Params...& pq) {
    std::priority_queue<Type, Params...> pq_to_print = pq;
    while (!pq_to_print.empty()) {
        os << pq_to_print.top() << " ";
        pq_to_print.pop();
    } // while
} // print_priority_queue()

// works on std::priority_queue<int32_t>, on a min-heap using std::greater<int32_t>,
// and on std::priority_queue<std::string, std::deque<std::string>, std::greater<std::string>>
```

1.9.3 Template Metaprogramming

Marked as extra material in the notes. (NOT REQUIRED)

- **Template metaprogramming** uses templates themselves to perform computation during compilation, moving work off of runtime. It effectively turns the type system into a separate functional language run by the compiler.

```
// runtime version
uint64_t factorial(uint32_t n) {
    if (n <= 1) { return 1; }           // base case
    return n * factorial(n - 1);        // recursive step
} // factorial()
```

```
// compile-time version using template specialization
template <uint64_t N>
struct Factorial {
    static constexpr uint64_t value = N * Factorial<N - 1>::value;
};

template <> // specialization for the base case 0! = 1
struct Factorial<0> {
    static constexpr uint64_t value = 1;
};

int main() {
    constexpr uint64_t five_factorial = Factorial<5>::value; // computed by the compiler
} // main()
```

- Factorial<5> expands to 5 * Factorial<4>::value, then 5 * (4 * Factorial<3>::value), and so on down to the Factorial<0> specialization. The compiler unrolls the recursion and does the multiplication while compiling.
- The <type_traits> header (C++11) provides compile-time type queries and transformations, useful when templated code needs different logic per type.

```
#include <type_traits>

struct Student {
    std::string name;
    int32_t age;
    Student(const std::string& name_in, int32_t age_in) : name{name_in}, age{age_in} {}
};

template <typename T>
void print(T value) {
    // the _v suffix is a C++17 shortcut for ::value
    if constexpr (std::is_integral_v<T>) { std::cout << "Integer: " << value << "\n"; }
    else if constexpr (std::is_floating_point_v<T>) { std::cout << "Float: " << value << "\n"; }
    else if constexpr (std::is_same_v<T, Student>) { std::cout << "Hello " << value.name << "!\n"; }
    else { std::cout << "Other: " << value << "\n"; }
}

int main() {
    print(42);           // Integer: 42
    print(3.14);         // Float: 3.14
    print("hello");      // Other: hello
    Student s{"Bob", 21};
    print(s);            // Hello Bob!
} // main()
```

Remark: if constexpr discards the unused branches at compile time rather than runtime, so each type gets specialized code with no runtime overhead from the checks.

1.9.4 SFINAE and Template Overloading

Marked as extra material in the notes. NOT REQUIRED

- **SFINAE** = *Substitution Failure Is Not An Error*. If the compiler cannot substitute the template arguments into a candidate, it does not immediately error out. It keeps looking at other candidates and only errors if none work.
- This makes conditional template overloading possible: a substitution failure can be triggered on purpose when a type does not meet a requirement.

```
#include <type_traits>

// Version 1: integral types only
template <typename T>
// the _t suffix is a C++17 shortcut for ::type
typename std::enable_if_t<std::is_integral_v<T>, void> print(T x) {
    std::cout << "Integral: " << x << '\n';
}

// Version 2: floating-point types only
template <typename T>
typename std::enable_if_t<std::is_floating_point_v<T>, void> print(T x) {
    std::cout << "Floating point: " << x << '\n';
}

// Version 3: fallback for everything else
template <typename T>
typename std::enable_if_t<!std::is_integral_v<T> && !std::is_floating_point_v<T>, void> print(T x) {
    std::cout << "Other type: " << x << '\n';
}

int main() {
    print(42);           // Integral: 42
    print(3.14);         // Floating point: 3.14
    print("hello");      // Other type: hello
} // main()
```

- For `print(42)`, `is_integral` is true and `is_floating_point` is false, so only version 1 substitutes successfully (`std::enable_if<false, void>::type` does not exist, which is the substitution failure).
- Only **one** version may be viable per call. If the compiler cannot pick a single best match, the call is ambiguous and compilation fails.

Remark: C++20 **concepts** do the same job with clearer syntax and better error messages:

```
template <std::integral T>      void print(T x) { std::cout << "Integral: " << x << '\n'; }
template <std::floating_point T> void print(T x) { std::cout << "Floating point: " << x << '\n'; }
template <typename T>          void print(T x) { std::cout << "Other type: " << x << '\n'; }
```

1.10 Prefix and Postfix Incrementation Operators CORE

- `operator++` and `operator--` change a variable by one, but position matters.
- **Prefix** (`++i`): increments first, then returns the **new** value.

```
int32_t i = 281;
int32_t j = ++i;           // increment i, then store in j
std::cout << j << std::endl; // prints 282
```

- **Postfix** (`i++`): returns the **original** value, and the increment happens at the very end.

```
int32_t i = 281;
int32_t j = i++;           // store in j, then increment i
std::cout << j << std::endl; // prints 281
```

- Same behavior for decrementation:

```
int32_t i = 281;           int32_t i = 281;
int32_t j = --i;           int32_t j = i--; // 281
```

- **Best practice:** if you do not need the old value, prefer prefix. Compilers often optimize the difference away, but not always, and prefix is never less efficient than postfix.

1.11 Loops CORE

1.11.1 For and While Loops

```
for (init; condition; update) {
    // code
}

while (condition) {
    // code
}
```

- In a for loop: `init` runs once before the loop, `condition` decides whether the loop keeps running, and `update` runs after each pass through the body. The loop ends when `condition` is false.
- A while loop repeats its block while its condition is true.
- Converting a for loop to a while loop: move the initialization above the loop and the update to the end of the body.

```
// for loop
for (int32_t i = 0; i < 10; ++i) {
    // code
} // for i

// converted while loop
int32_t i = 0;
while (i < 10) {
    // code
    ++i;
} // while
```

- Converting a while loop to a for loop: omit the initialization statement.

```
// while loop
while (*ptr++ != nullptr) {
    // code
    ++counter;
} // while

// converted for loop
for (; *ptr != nullptr; ++ptr, ++counter) {
    // code
} // for
```

1.11.2 Range-Based For Loop

- Loops over every element in a collection:

```
for (individual element in collection : collection) { ... }
```

```
std::vector<int32_t> vec = {1, 2, 3, 4, 5};

// traditional loop with an index counter
for (size_t i = 0; i < vec.size(); ++i) {
    vec[i] *= 2;
} // for i

// range-based, visits each element directly
for (int32_t& x : vec) {
    x *= 2;
} // for x

// same thing with auto
for (auto& x : vec) {
    x *= 2;
} // for x
```

- Two parts separated by a colon: the variable holding the current element, and the collection.
- **Important:** to modify elements, the loop variable must be a **reference** (&). Without it the loop copies each element, and modifications hit the copy, leaving the container unchanged.

1.11.3 Do-While Loop

- A **post-test** loop: the body always runs at least once, and the condition is checked at the end.

```
do {
    // body of loop
} while (condition);
```

```
int32_t num = 281;
do {
    std::cout << "EECS " << num << std::endl;
    ++num;
} while (num < 280);
// prints "EECS 281" once, then the condition fails and the loop ends
```

1.11.4 The break Keyword

- `break` terminates the **nearest enclosing loop**. Execution resumes right after that loop's body.

```
while (/* condition */) {
    // do stuff
    if (/* condition for break */) {
        break; // while loop terminates immediately
    } // if
    // do more stuff
} // while
// more code here
```

- Useful to end an otherwise infinite loop or to stop early.
- In nested loops, only the innermost enclosing loop is exited:

```
while (true) {
    for (int32_t i = 0; i < 5; ++i) {
        if (i == 3) { break; } // exits the for loop only
        std::cout << i << " ";
    } // for i
    std::cout << "PotatoBot" << std::endl;
} // while
// prints "0 1 2 PotatoBot" forever
```

1.11.5 The continue Keyword

- `continue` skips the rest of the current iteration and starts the next one. The loop itself keeps running.

```
for (int32_t i = 1; i <= 5; ++i) {
    if (i == 3) { continue; }
    std::cout << i << " ";
} // for i
// prints "1 2 4 5 "
```

1.12 Inheritance and Polymorphism PROJECT USE

You will not be required to use polymorphism on your own, but it appears in starter files, so it is worth refreshing.

1.12.1 Inheritance

- **Object-oriented programming (OOP)** solves problems using objects and their methods. Its four principles:
 1. **Encapsulation:** combining data and methods into a single unit. In C++ this is a class with private member variables holding the state and public member functions that modify it.
 2. **Abstraction:** hiding internal implementation details and exposing only what is needed. You call `std::vector::erase()` without knowing how it works.
 3. **Inheritance:** a class reusing the interface or functionality of another class.
 4. **Polymorphism:** covered in 1.12.2.
- Inheritance exists because many objects in an OOP design are similar but not identical. It lets them share common logic while still having distinct behavior.

```
class UMPerson {
    std::string name;
    int32_t id;
    int32_t age;
public:
    UMPerson(std::string name_in, int32_t id_in, int32_t age_in)
        : name{name_in}, id{id_in}, age{age_in} {}

    std::string get_name() const { return name; }
    int32_t get_id() const { return id; }
    int32_t get_age() const { return age; }
};
```



- Students and professors are both a UMPerson (name, UMID, age) but differ: students have a GPA, grade level, classes taken, and graduation year; professors have classes taught, years at the university, a salary, and a rank or title.
- The class with the shared functionality is the **base class**. Classes built from it are **derived classes**.
- Syntax: a colon after the derived class name, then `public`, then the base class name.

```
class Student : public UMPerson {
    std::vector<std::string> classes_taken;
    double gpa;
    int32_t grade_level;
    int32_t graduation_year;
};
```

- With **public inheritance**, the derived class gets everything the base has and can directly access any non-private member of the base.
- Private members of the base are **inherited but not directly accessible** by the derived class, which is why the getter functions exist.

Remark: other inheritance types

- **private inheritance:** replace `public` with `private`. The derived class cannot directly access public members of the base.
 - **protected:** a protected member cannot be accessed from outside the class (like private), but **is** directly accessible from any derived class. If `name` in UMPerson were protected, Professor could touch it directly while unrelated code could not.
 - These alternatives are much less common.
- A derived class constructor must initialize the base class it derives from. Since the inherited members are private in UMPerson, Student cannot set them directly and must invoke the base constructor in its member-initializer list:

```
Student(std::string name_in, int32_t id_in, int32_t age_in, double gpa_in,
        int32_t grade_in, int32_t year_in)
    : UMPerson{name_in, id_in, age_in}, // base class constructor first
      gpa{gpa_in}, grade_level{grade_in}, graduation_year{year_in} {}
```

- A derived constructor **always** invokes a base constructor. If you do not name one explicitly, the base class default constructor is called implicitly.

```
class Professor : public UMPerson {
    std::vector<std::string> classes_taught;
    std::string department;
    std::string title;
    double salary;
    int32_t years_taught;
    bool tenured;
public:
    Professor(std::string name_in, int32_t id_in, int32_t age_in, std::string dep_in,
              std::string title_in, double salary_in, int32_t years_in, bool tenured_in)
        : UMPerson{name_in, id_in, age_in}, department{dep_in}, title{title_in},
          salary{salary_in}, years_taught{years_in}, tenured{tenured_in} {}
};
```

1.12.2 Polymorphism

- **Polymorphism** allows a derived class object to be used wherever a base class object is expected. (Strictly this is *subtype polymorphism*, which is what the term usually means.)

- A base-type pointer or reference may refer to a derived object:

```
UMPerson* student_ptr = &my_student;    // okay, a Student is a UMPerson
UMPerson& student_ref = my_student;      // okay
UMPerson* prof_ptr = &my_professor;      // okay
UMPerson& prof_ref = my_professor;       // okay
```

- The reverse is not implicitly allowed, because not every UMPerson is a Student:

```
UMPerson myUMP{"Bob", 12345678, 21};
Student* student_ptr = &myUMP;           // not okay
Student& student_ref = myUMP;            // not okay
Professor* prof_ptr = &myUMP;            // not okay
Professor& prof_ref = myUMP;             // not okay
```

- To go from base to derived you must cast explicitly with `static_cast<>` or `dynamic_cast<>` (section 1.13).

Remark: object slicing

```
UMPerson myUMP = myStudent;    // not okay
```

Even though a `Student` derives from `UMPerson`, a `Student`'s value does not fit into a `UMPerson` object. Only the members defined by `UMPerson` are copied, and the extra `Student` members are "sliced off."

Example: ostream

```
      ostream
     /      \
ostream    ostream
```

- `operator<<` expects an `ostream`, but `ostream` is a base class of other output stream types, so polymorphism lets it work on an `ostream`:

```
std::ostream& operator<<(std::ostream& os, const Point& pt) {
    os << "(" << pt.x << ", " << pt.y << ")";
    return os;
} // operator<<()

int main() {
    Point pt{3, 4};
    std::ostream os;
    os << pt;
    std::cout << os.str() << std::endl;    // works on ostream
} // main()
```

Containers of base class pointers, and the virtual keyword

```
class UMPerson {
public:
    std::string name;
    UMPerson(std::string name_in) : name{name_in} {}
    void print_name() const { std::cout << "Person " << name << std::endl; }
};

class Student : public UMPerson {
public:
    Student(std::string name_in) : UMPerson{name_in} {}
    void print_name() const { std::cout << "Student " << name << std::endl; }
};

class Professor : public UMPerson {
public:
    Professor(std::string name_in) : UMPerson{name_in} {}
    void print_name() const { std::cout << "Professor " << name << std::endl; }
};

int main() {
    Student a = Student{"Alice"};
    Student b = Student{"Bob"};
    Professor c = Professor{"Cathy"};
    Professor d = Professor{"Dave"};
    std::vector<UMPerson*> vec = { &a, &b, &c, &d };
    for (const auto& person : vec) {
        person->print_name();
    } // for
} // main()
```

Output (wrong, the base version runs for everyone):

```
Person Alice
Person Bob
Person Cathy
Person Dave
```

- Fix with `virtual`. When a virtual function is called, the compiler picks the version based on the object's type **at runtime (dynamic binding)** instead of its type at compile time (**static binding**).
- `virtual` is only needed in the **base** class and is optional in derived classes. It can only be prepended to a function definition inside a class; a function defined outside the class does not need it.
- In derived classes use `override`, which tells the compiler the function is meant to override a base member function. If it overrides nothing, the compiler issues an error, which catches bugs.

```

class UMPerson {
public:
    std::string name;
    UMPerson(std::string name_in) : name{name_in} {}
    virtual void print_name() const { std::cout << "Person " << name << std::endl; }
};

class Student : public UMPerson {
public:
    Student(std::string name_in) : UMPerson{name_in} {}
    void print_name() const override { std::cout << "Student " << name << std::endl; }
};

class Professor : public UMPerson {
public:
    Professor(std::string name_in) : UMPerson{name_in} {}
    void print_name() const override { std::cout << "Professor " << name << std::endl; }
};

```

Output (correct):

```

Student Alice
Student Bob
Professor Cathy
Professor Dave

```

1.13 Casting PROJECT USE

- **Casting** converts data from one type to another.
- **Implicit conversions** are done automatically by the compiler:

```

int32_t a = 281;
double b = a; // int implicitly converted to double

```

- **Explicit conversions** are requested by the programmer. C++ has four explicit casts: `static_cast<>`, `dynamic_cast<>`, `reinterpret_cast<>`, `const_cast<>`.

1.13.1 Static Cast

- The standard, well-behaved cast for converting between two compatible types.
- Preferred for automatic conversions and **narrowing conversions** (going from a wider range to a smaller one, like `double` to `int`).
- Advantages: readable, expresses clear intent, and **safe**, since the compiler errors on an incompatible conversion.

```

// valid
double a = 281.99;
int32_t b = static_cast<int32_t>(a); // b is 281, narrowing truncates the decimals

// invalid
class MyClass { char a = 'a'; }; mc;
char mc_a = static_cast<char>(mc); // compile error

```

1.13.2 Dynamic Cast

Marked as extra material in the notes. NOT REQUIRED

- Used for type-safe **downcasting**: converting a base class pointer or reference to a derived one.
- The program performs a **runtime check** to see whether the conversion is possible. On failure it returns `nullptr`.
- Really only applicable when downcasting a polymorphic type.

```

void process_person(UMPerson& person) {
    if (auto as_student = dynamic_cast<Student*>(&person)) {
        std::cout << "Received a student named " << as_student->name << std::endl;
    } // if
    else if (auto as_professor = dynamic_cast<Professor*>(&person)) {
        std::cout << "Received a professor named " << as_professor->name << std::endl;
    } // else if
} // process_person()

int main() {
    Student a{"Alice"};
    Professor b{"Bob"};
    process_person(a); // "Received a student named Alice"
    process_person(b); // "Received a professor named Bob"
} // main()

```

1.13.3 Reinterpret Cast

Marked as extra material in the notes. NOT REQUIRED

- Takes the memory representation of an object and reinterprets those bits as another type.

```
struct MyClass {
    int16_t a;
    int16_t b;
};

int main() {
    double d = 281.37;
    uint64_t ptr_addr = reinterpret_cast<uint64_t>(&d); // pointer reinterpreted as integer
    std::cout << ptr_addr << std::endl;

    int32_t num = 24248601;
    MyClass* mc = reinterpret_cast<MyClass*>(&num); // int bits reinterpreted as MyClass
    std::cout << mc->a << std::endl; // prints 281 (system dependent)
    std::cout << mc->b << std::endl; // prints 370 (system dependent)
} // main()
```

- Because it reinterprets physical bits, it is very dangerous and can easily cause misbehavior. Mentioned only for completeness. Do not use it in your code.

1.13.4 Const Cast

Marked as extra material in the notes. NOT REQUIRED

- Removes constness from a pointer or reference that refers to an object that is **not** itself const.

```
int main() {
    int32_t i = 0;
    const int32_t& i_ref = i;
    // i_ref = 281; // not allowed, i_ref is const
    const_cast<int32_t&>(i_ref) = 281; // const cast, then assign
    std::cout << i_ref << std::endl; // prints 281
    std::cout << i << std::endl; // prints 281
    // i_ref = 370; // still not allowed, i_ref is still const

    int32_t j = 0;
    const int32_t* j_ptr = &j;
    // *j_ptr = 281; // not allowed
    *const_cast<int32_t*>(j_ptr) = 281;
    std::cout << *j_ptr << std::endl; // prints 281
    std::cout << j << std::endl; // prints 281
} // main()
```

- Dangerous if used incorrectly, and usually a sign of poorly designed code. Do not use it in this class.

1.13.5 C-Style Casts

Marked as extra material in the notes. NOT REQUIRED

```
double a = 281.99;
int32_t b = (int32_t)a; // b is 281
```

- **Avoid C-style casts in C++.** A C-style cast can behave like a static cast, a const cast, or even a reinterpret cast depending on the types involved, which can lead to dangerous or undefined behavior if you cast the wrong thing.
- `static_cast<>` catches incompatible conversions at compile time, is more readable, and makes conversions easy to spot in the code.

1.14 Exception Handling PROJECT USE

You will not be required to throw exceptions for any assignment, although using them can make code cleaner. Knowing how to handle them is useful for upper-level courses.

1.14.1 Try-Catch Blocks

- An **exception** is an event issued during execution when erroneous conditions come up at runtime. Advantages over other error handling:
 - Error handling code stays separate from normal program code, improving readability and maintainability.
 - Exceptions are versatile: a method can throw many exceptions but choose which ones to handle. Unhandled ones propagate upward to a caller that can handle them.
 - Exceptions can be grouped into a hierarchy through inheritance, so similar errors can be handled together.
- Three keywords: `try`, `catch`, `throw`.

```
int main() {
    try {
        // code that could potentially throw an exception
    } // try
    catch (const EXCEPTION_NAME& ex) {
        // code that handles exceptions thrown within the try block
    } // catch
} // main()
```

- A try block may have multiple catch blocks. When an exception is thrown, the catch blocks are checked **top to bottom**, and execution

jumps to the first one that can handle it.

- After that catch block finishes, execution continues past the whole try-catch. Any remaining code in the try block is **skipped**.
- If no catch block matches, the exception propagates to the caller of the function that threw it, and the search repeats. If it propagates past `main()` unhandled, the program terminates and the user is told an error occurred.

1.14.2 Exception Inheritance

- Technically anything can be thrown:

```
int main() {
    try {
        int32_t fav_class;
        std::cout << "What is your favorite class?" << std::endl;
        std::cin >> fav_class;
        if (fav_class == 281) {
            std::cout << "Of course! Who doesn't love EECS 281?" << std::endl;
        } // if
        else {
            throw (fav_class);
        } // else
    } // try
    catch (int32_t wrong_class) {
        std::cout << "You said your favorite class is " << wrong_class << std::endl;
        std::cout << "YOU ARE WRONG" << std::endl;
    } // catch
} // main()
```

- The standard library hierarchy (as of C++17), all deriving from `std::exception`:

```
std::exception
|
|-- std::bad_alloc --> std::bad_array_new_length
|-- std::bad_cast --> std::bad_any_cast
|-- std::bad_exception
|-- std::bad_function_call
|-- std::bad_optional_access
|-- std::bad_typeid
|-- std::bad_variant_access
|-- std::bad_weak_ptr
|
|-- std::logic_error (logic error exceptions)
|   |-- std::domain_error
|   |-- std::future_error
|   |-- std::invalid_argument
|   |-- std::length_error
|   |-- std::out_of_range
|
|-- std::runtime_error (runtime error exceptions)
|   |-- std::overflow_error
|   |-- std::range_error
|   |-- std::regex_error
|   |-- std::system_error --> std::filesystem::filesystem_error
|   |-- std::underflow_error
|   |-- std::ios_base::failure
```

- Every exception derives from `std::exception`, which implements `what()`, a member function returning a message identifying the exception.

```
int main() {
    try {
        throw std::runtime_error{"descriptive message here"};
    } // try
    catch (const std::runtime_error& ex) {
        std::cerr << ex.what() << std::endl; // prints "descriptive message here"
    } // catch
} // main()
```

- One you are likely to see: `std::bad_alloc`, thrown when `new` fails to allocate the requested memory.

```
try {
    char* arr = new char[STACK_SIZE]; // throws bad_alloc if not enough memory
} // try
catch (const std::bad_alloc& ex) {
    std::cerr << "bad_alloc caught: " << ex.what() << std::endl;
} // catch
```

- You can define **custom exceptions** that inherit from `std::exception`. `what()` is virtual, so you can override it:

```
class EECS281Exception : public std::exception {
    std::string msg;
public:
    EECS281Exception(const std::string& msg_in) : msg{msg_in} {}

    const char* what() const noexcept override {
        return msg.c_str();
    } // what()
};

// throw EECS281Exception{"YOU ARE WRONG"}; then catch (const EECS281Exception& ex)
// and print ex.what() to get "YOU ARE WRONG"
```

Remark: the noexcept keyword

A noexcept function promises never to throw. It should not throw on its own, use `new`, call a library function that could throw, perform arithmetic that could overflow or underflow, or do anything else that could raise an exception. If such a function does throw anyway, the normal try-catch procedure is skipped and the program terminates immediately.

- Because catch clauses are checked in order, **derived exception types must be caught before their base types**:

```
try {  
    /* ... code that could throw any type of logic error ... */  
}  
catch (const std::out_of_range& ex) { // derived, must come first  
    /* ... */  
}  
catch (const std::logic_error& ex) { // base  
    /* ... */  
}
```

- If the order were reversed, the `logic_error` block would also catch `out_of_range` (since it derives from `logic_error`), and the `out_of_range` block would never run.
- An ellipsis (...) as the catch parameter catches **anything**, so it belongs below all other catch blocks:

```
try { /* ... */ }  
catch (const std::logic_error& ex) { /* ... */ }  
catch (const std::runtime_error& ex) { /* ... */ }  
catch (...) { /* handles anything else */ }
```

- A catch block may **rethrow** the exception it caught, which is useful to do extra work locally while still letting the exception propagate upward:

```
try { /* ... */ }  
catch (const std::exception&) {  
    /* ... do some additional work ... */  
    throw; // original exception is rethrown to the caller  
}
```

You may still meet exceptions through the EECS 281 autograder even if you never throw one. A test case that exceeds the maximum allowable memory limit forces the program to throw `std::bad_alloc`, which shows up in the autograder output alongside the failed test case, its runtime, and its memory usage.